

Introduction to Python

Week 2

Programme for Today

1. Augmented Assignment Operators
2. Dictionaries, lists and sets
3. Conditions
4. Control structures
 1. Conditional statements and expressions
 2. For-loops
 3. Looping over collections of data
 4. While-loops
5. Other
 1. Conversion of data types
 2. Truthy and Falsy
 3. Common homework difficulties

The Augmented Assignment Operators

- $a = a + 1$
 $a += 1$
- $a = a - 2$
 $a -= 2$
- $a = a * 3$
 $a *= 3$
- $a = a / 4$
 $a /= 4$
- $a = a // 5$
 $a //= 5$
- $a = a \% 6$
 $a \% = 6$

Dictionaries: Introduction

- Recall from last week: dictionaries are sets of key: value pairs:

- {key1: value1, key2: value2, etc}

- Why use dictionaries?

1. Using dictionaries, makes your program more readable

- Compare:

- capitals = {'Estonia': 'Tallinn', 'Belgium': 'Brussels', 'France', 'Paris'}

```
print(capitals['Belgium'])
```

- countries = ['Estonia', 'Belgium', 'France']

```
capitals = ['Tallinn', 'Brussels', 'Paris']  
print(capitals[countries.index('Belgium')])
```

- capitals = ['Estonia', 'Tallin', 'Belgium', 'Brussels', 'France', 'Paris']

```
print(capitals[capitals.index('Belgium') + 1])
```

Dictionaries: Introduction

2. Dictionaries are fast

- In Python there often is trade-off between speed and flexibility
- To make dictionaries fast, keys in a dictionary must be unique and immutable

- #

Python actually wants a key to be hashable, and while there is a difference between hashable and immutable, it is enough for this course to assume it is the same. If you are interested you can google to find out more

- Dictionaries have no order while lists for example have
 - `print ({1:1, 2:2} == {2:2, 1:1})`
 - `print ([1,2] == [2,1])`
- This also means that you cannot sort dictionaries

Using dictionaries

- How to create an empty dictionary:
 - `capitals = {}`
 - `capitals = dict()`
- How to create a filled dictionary:
 - `capitals = {'Andorra': 'Andorra la Vella', 'Belgium': 'Brussels'}`
 - `l1 = ['Andorra', 'Belgium']`
`l2 = ['Andorra la Vella', 'Brussels']`
`capitals = dict(zip(countries, capitals))`
- How to select by key:
 - `print(capitals['Belgium'])`
- How to delete a key:value pair:
 - `del(capitals['Belgium'])`

Using dictionaries

- How to insert new key:value pairs:
 - `capitals['Netherlands'] = 'The Hague'`
 - `capitals.update({'Netherlands' : 'The Hague', 'Belgium': 'Brussels'})`
- How to update your dictionary:
 - `capitals['Netherlands'] = 'Amsterdam'`
 - `capitals.update({'Netherlands' : 'Amsterdam', 'Belgium': 'Brussels'})`
- Updating/inserting is a bit tricky: sometimes you must be sure that you are updating an existing key/value pair, and not inserting a new one, so:
 - ```
if 'Netherlands' in capitals:
 capitals['Netherlands'] = 'The Hague'
else:
 print("Country doesn't exist")
```

# Lists vs dictionaries

- Lists are sequences of values [value1, value2, etc]
  - *Sequences* are collections that are ordered
- Lists, tuples, and strings are sequences
- Dictionaries and sets are not sequences
- These sequences and not-sequences are both called *collections*
  - `print ([1,2] == [2,1])` # Onscreen: False
  - `print ((1,2) == (2,1))` # Onscreen: False
  - `print ('12' == '21')` # Onscreen: False
  - `print ({1,2} == {2,1})` # Onscreen: True
  - `print ({1:1, 2:2} == {2:2, 1:1})` # Onscreen: True



# Using lists

- How to create an empty list:
  - `countries = []`
  - `countries = list()`
- How to create a filled list:
  - `countries = ['Andorra', 'Belgium']`
- How to select by index:
  - `print(countries[1])`
- How to delete a value:
  - `del(countries[1])`

# Using lists

- Inserting new value:

- `countries += ['Netherlands', 'Belgium']`
- `countries.append('Netherlands')`

- Inserting can be a bit tricky: sometimes you want to be sure that a certain value is not already in the list, so:

- ```
if 'Netherlands' not in countries:
    countries += ['Netherlands']
else:
    print('Country already in list')
```

- How to update your list:

- `countries[-1] = 'Holland'`

Sets

- Sets are collections with values {value1, value2, etc} , where each values is unique
 - `print ({1, 2, 2} == {2,1})` # Onscreen: True
 - `print (len({1, 2, 2}) == 2)` # Onscreen: True
- How to create an empty set:
 - `countries = set()`
 - `countries = {}` Would this work?
- How to create a filled set:
 - `countries = {'Andorra', 'Belgium'}`
- How to select by index:
 - Not possible
- How to delete a value:
 - `countries.remove('Andorra')`
- Inserting a new value:
 - `countries.add('Andorra')`
 - `countries.update({'Andorra', 'Greece'})`

Conditions: Introduction

- Conditions are expressions that lead to `True` or `False`
- Comparison operators (`<`, `<=`, `>`, `>=`, `==`, `!=`): compare two values, the result is either `True` or `False`
- Inclusion check (`in`): check whether a value is in a collection. In case of a dictionary the inclusion checks whether the value is one of the keys.

• <code>print(1 in [1,2,3])</code>	<code># Onscreen: True</code>
• <code>print(1 in (1,2,3))</code>	<code># Onscreen: True</code>
• <code>print('1' in '123')</code>	<code># Onscreen: True</code>
• <code>print(1 in '123')</code>	<code># Onscreen: TypeError</code>
• <code>print(1 in {1: 4,2: 3})</code>	<code># Onscreen: True</code>
• <code>print(4 in {1: 4,2: 3})</code>	<code># Onscreen: False</code>
• <code>print(1 in {1,2,3})</code>	<code># Onscreen: True</code>

Conditions: boolean operators

- Use Boolean operators to create more complex conditions
- Boolean operators can be used to connect comparisons: `and`, `or`
 - ```
print(True and True, True or True) # Onscreen: True True
print(True and False, True or False) # Onscreen: False True
print(False and True, False or True) # Onscreen: False True
print(False and False, False or False) # Onscreen: False, False
```
- Boolean operators can be used to negate a Boolean value: `not`
  - ```
print(not True)                # Onscreen: False
print(not False)               # Onscreen: True
```
- With this small set of building blocks, you can construct very complex conditions

Conditions: tricky example

- Using conditions can be tricky
- For example, say you want to get True only if both variables `a` and `b` are either 1 or 2:
 - ```
a=1
b=3
print(a==1 or a==2 and b==1 or b==2)
```
  - Does it make sense? To a human it does, but not to Python
- What will Python make of the above code (in nerdy language: how does Python evaluate this code)?
  - Step 1: Python replaces the comparisons by their Boolean values
  - Result: 

```
print(True or False and False or False)
```
  - Step 2: Python executes the `and`
  - Result: 

```
print(True or False or False)
```
  - Step 3: Python executes the `or`'s from left to right
  - Result: 

```
print(True) #Onscreen: True
```

# Conditions: tricky example

- This is clearly not what you want -> solution: in case of doubt, use round brackets:
  - ```
a=1
b=3
print((a==1 or a==2) and (b==1 or b==2))    #Onscreen: False
```
- An even nicer solution (applicable in this case) is using the inclusion operator:
 - ```
a=1
b=3
print(a in [1,2] and b in [1,2]) #Onscreen: False
```
- The whole problem about what Python does first, and what it does second etc, is called the *problem of precedence*
  - As in the above example, you can often avoid the problem, but in case you are interested there's a detailed overview of precedence in the official Python manual: <https://docs.python.org/3/reference/expressions.html#operator-precedence>

# Why/when to use conditions

- Now we know how to create conditions (expressions that evaluate into True or False), what are we going to do with them?
  - Use in conditional statements
  - Use in loops



# Conditions in conditional statements

- Conditional statements check for a given condition and conditionally on the outcome, certain blocks of code will be executed
- Example:
  - ```
if condition_1:  
    expression_1  
elif condition_2:  
    expression_2  
    expression_3  
else:  
    expression_4
```
- You always have one if (always at the start), at most one else (always at the end), and as many elif's in between as you want
- It's important to note the indentation
 - After a colon you always have to indent your code
 - All code that belongs together should get the same indentation

Conditional expressions

- Conditional expressions are useful short-cuts, but they come with a trade-off
 - They enhance your overview (make your code more concise)
 - But they make your code also a bit more complex
- Compare:
 - Conditional statement:

```
if a > 0:  
    b = a  
else:  
    b = 0
```
 - Conditional expression: `b = a if a > 0 else 0`
 - (Note from Mathijs: `b = max(0, a)`)
- The more experienced you become in programming, the more you will probably choose to use these kind of short-cuts
 - In future lectures we will discuss comprehensions, and those also make your code more concise

Loops

- for-loops
- while-loops

For-loops

- `for variable in sequence:`
 `expression`
- Python executes the expression a given number of times or for a given number of elements
 - You can manipulate this default behavior with `break` and `continue`
 - `break`: skip to the first statement after the loop
 - `continue`: go back to the statement with the `for` keyword
- Before each execution of the expression, the variable is filled with one element after the other from the sequence, until the sequence is exhausted

For-loops: examples

- You want to get the total of a series of numbers

- ```
total = 0
for number in [1, 2, 3, 3, 5, 7]:
 total += number
print (total) # Onscreen: 21
```

- You want to get the total of a series of numbers, but skip potential string values

- ```
total = 0
for number in [1, 2, '3', 3, 5, 7]:
    if type(number) == str:
        continue
    total += number
print (total) #Onscreen: 18
```

For-loops: examples

- Note: you are not forced to use the variable created by the for-loop :
 - ```
for number in range(5):
 print ('Hello') #Onscreen: Hello Hello Hello Hello Hello
```
- If you want to make the above code more readable, the following is standard practice:
  - ```
for _ in range(5):  
    print ('Hello') #Onscreen: Hello Hello Hello Hello Hello
```

Looping over dictionaries

- You can only loop over sequences, but dictionaries are no sequences. So how to loop over dictionaries?

- You can create sequences from dictionaries

- Consider for example the following dictionary:

```
capitals = {'Andorra': 'Andorra la Vella', 'Belgium': 'Brussels'}
```

- `capitals.keys()` gives you a sequence of dictionary keys
- `capitals.values()` gives you a sequence of dictionary values
- `capitals.items()` gives you a sequence of tuples with for each pair the key and the value

Looping over dictionaries

- You can use the sequences mentioned on the previous slide in a for loop:
 - `capitals = {'Andorra': 'Andorra la Vella', 'Belgium': 'Brussels'}`
 - `for key in capitals.keys():`
 `print(key)` `# Onscreen: Andorra Belgium`
 - `for key in capitals.values():`
 `print(key)` `# Onscreen: Andorra la Vella Brussels`
 - `for key, value in capitals.items():`
 `print(key, value)` `# Onscreen: Andorra Andorra la Vella`
 `# Belgium Brussels`
- The following lines of code do the same:
 - `for key in capitals:`
 - `for key in capitals.keys():`

Looping: enumerate()

- Consider for example you have the following list:

```
countries = ['Andorra', 'Belgium']
```

- Say you want to loop over this list, but for each of the values you also need its index, then you should use the enumerate() function
 - ```
for index, country in enumerate(countries, 1):
 print('country ' + country + ' has index: ' + str(index))
 # Onscreen: country Andorra has index: 1
```
  - The second parameter of the enumerate function, gives a start value for the index. The default is 0

# Looping: zip()

- Consider for example you have the following lists:
  - `countries = ['Andorra', 'Belgium', ...]`
  - `capitals = ['Andorra la Vella', 'Brussels', ...]`
- Say you want to loop over these lists in tandem, so for every country you also need the capital, then you should use the `zip()` function
  - ```
for country, capital in zip(countries, capitals):  
    print('country ' + country + ' has capital: ' + capital)  
# Onscreen: country Andorra has capital: Andorra la Vella, ...
```
- Note:
 - You can zip more than 2 sequences
 - It is best you zip sequences of equal length; Python has a way to deal with sequences of unequal lengths, but this can be a bit messy
 - You can zip sequences of different types

While-loop

- `while condition:`
 `expression`
- Executes the lines in the expression, if the condition is `True`
- Python first checks the condition before the expression is evaluated
- You can also use `break` and `continue`
- **Watch out for infinite loops!**
 - For example, on a (free) PythonAnywhere account
 - Use `Ctrl + C`, if you accidentally get into one
- If it all goes wrong, you must wait 24 hours
- With for-loops you know (most of the time) beforehand how much data you must process at most. Not with a while-loop

While-loop: example

- A program asks the user to enter numbers, returns the sum of all odd numbers and stop with 100

- ```
total = 0
number = int(input('give a number, or 100 to stop '))
while number != 100:
 if number % 2 == 1:
 total += number
 number = int(input('give number, or 100 to stop '))
print (total)
```

- **Adding** break and continue:

- ```
total = 0
while True:
    number = int(input('give a number, or 100 to stop '))
    if number == 100:
        break
    if number % 2 == 0:
        continue
    total += number
print (total)
```

- Which style do you prefer?

The walrus operator := (introduced in Python 3.8)

- The standard assignment operator (=) assigns a name to an object, but doesn't evaluate to a value
- The walrus operator does evaluate to a value, and it evaluates to the value of the newly created object
- The following works in some computer languages, but not in Python

- ```
total = 0
while (number = int(input('give a number, or 100 to stop '))) != 100 :
 if number % 2 == 1:
 total += number
print (total)
```

- But now you can, using the walrus operator

- ```
total = 0
while (number := int(input('give a number, or 100 to stop '))) != 100 :
    if number % 2 == 1:
        total += number
print (total)
```

- Another useful application, we will see when discussing comprehensions
- NB! Uitleggen verschil statement en expressie

Solving question 9 with the Walrus

- Assume that you already have one variable called 'x', which is a list that contains at least 8 integer elements. Two elements of this list have the value 17.
- Print a list with all the elements between the two 17's. Both 17's should not be included. You can assume that there will only be two 17's in the list.
- For example:
- If `x = [21, 17, 29, 20, 17, 21, 38, 26]`, your program should print: `[29, 20]`
- ```
left = x.index(17) + 1
right = x.index(17, left)
print(x[left:right])
```
- ```
print(x[(left:=x.index(17)+1) : x.index(17, left)30])
```

For-loop vs while-loop

- Anything you can do with a for loop you can also do with a while-loop, but as we saw, you can even do more with a while-loop
- So, if you really don't know how to solve a looping problem with a for-loop, you can always use a while-loop

Conversion of datatypes

- Compared with other programming languages, Python is not very generous w.r.t. automatically converting datatypes
 - `print('1' + 1)` # Onscreen: error, the designers of Python could have chosen automatic conversion of the string '1' into integer 1
 - `print('1.0' + 1)` # Onscreen: error, the designers of Python could have chosen automatic conversion of the string '1.0' into float 1.0
 - `print(1232.count(2))` # Onscreen: error, the designers of Python could have chosen automatic conversion of the integer 123 into string '123' and answer the question
- If you want these conversions, you must make those conversions yourself
- Advantage: more control, less ambiguity, fewer surprises
 - In JavaScript `'1' + 2 + 3`, gives you `'123'` and not 6, you could be surprised as there is no error
- Disadvantage: more work
 - Compare `'page' + number` (JavaScript) with: `'page' + str(number)` (Python)

Truthy and Falsy: automatic conversion in conditions

- We said Python is not very generous w.r.t. automatically converting datatypes, but datatypes of variables used as a condition are the exceptions to the rule
- When used as a condition the following is seen as True or False:

False	True
[]	[1]
()	(1)
{}	{1:3}
{}	{1}
"	' '
0	2
0.0	2.0

Truthy and Falsy: example

- ```
l1 = [1, 2, 3, 4, 5, 6]
total = 0
while l1:
 total += l1[0]
 del l1[0]
print(total) # Onscreen: 21
```
- Looks very similar to using a for-loop, where you run over a collection until it is exhausted:
  - ```
l1 = [1, 2, 3, 4, 5, 6]
total = 0
for number in l1:
    total += number
print(total) # Onscreen: 21
```

Examples: explicit conversions

- `print(str(1) == '1') # Onscreen: True`
- `print(int('1') == 1) # Onscreen: True`
- `print(int('1a') == 1) # Onscreen: error`
- `print(float('1') == 1.0) # Onscreen: True`
- `print(float('1.0') == 1.0) # Onscreen: True`
- `print(tuple([1,2,3]) == (1,2,3)) # Onscreen: True`
- `print(tuple({1:3, 2:4}) == (1,2)) # Onscreen: True`
`print(list(range(1,5,2)) == [1, 3]) # Onscreen: True`
- `print(dict([1, 2])) # Onscreen: error`
- `print(dict([(1,3), (2,4)]) == {1: 3, 2:4})`
- **Some conversion functions:** `dict()`, `tuple()`, `list()`, `set()`, `int()`, `bool()`, `float()`

Homework week 1

- Try to understand the suggested solutions, that is good preparation for the exam
- In general, the building blocks from DataCamp should be enough to understand them
- If it is not clear, play around with the code in Pythonanywhere

Homework: Forgetting to save

- Say, you were sure to take the line with `x = 'Mary'` out, but you still fail the test -> the problem could be that you didn't save your new version. Actually, you can see under Solution being tested, what is being tested
- Not saving is the most common and maybe most annoying error

SOLUTION BEING TESTED

=====

```
x = 'Mary'
print('Good morning ' + x + '!')
```

TEST RESULT

=====

FAILED TEST!

Preconditions

x = 'Bernard'

Expected printout

Good morning Bernard!

Actual printout

Good morning Mary!

Homework: Look at the example

- Last part before `"""` is the example
 - The example can be very useful to understand the question. Sometimes you will understand the question fully after reading the example and/or after some runs of `test_answer`. So don't be worried if you don't understand the question at once, use all these tools to get a grip
 - You can try to replicate the example. In that case you must fill the variables like is done in the example and run your script outside the `test_answer` system. If you can replicate the example, you're likely to have solved it
- Below this part, you can find hints
 - Sometimes they tell you step-by-step what to do
 - Sometimes they give you just a clue about a part of the answer

`"""`

Assume that you already have one variable called 'x', which is a list that contains 4 elements.

Print a new list that contains elements of x in this order:
first - third - fourth.

If `x = ['r', 'q', 'u', 'x']`, then your program should print:

`['r', 'u', 'x']`

`"""`

Homework: Be careful with names

- `n1 = 100`
 `n2 = x/100`
 `n3 = n2 + 1`
 `n4 = n1*n2**7`
 `n5 = round(n4, 2)`
- This is a silly example, but a lot of errors occur because programmers forget which object was referred to by a certain name
 - These errors can be very difficult to find, as Python doesn't throw an error

Homework: Avoid using Python's reserved words

- ```
countries = ['Netherlands', 'Belgium']
capitals = ['Amsterdam', 'Brussels']
sorted = sorted(countries)
sorted2 = sorted(capitals) # Onscreen: TypeError: 'list'
object is not callable
```
- If you use more meaningful names like `sorted_countries` or `sorted_capitals` you will be safe, or if you go in the other direction `11`, `12` etc. also
- Remember that in Python built-in functions etc. are just objects with a name and you can accidentally attach that name to another object



# Question 2: working step-by-step versus inside out

- Assume that you already have a variable called 'x', which contains the percentage annual interest rate paid by a bank account as a nonnegative integer value.
- Suppose that the balance of the bank account is now 100.
- Print out what the balance of the bank account will be in 8 years.
- Before you print the new balance, use the round function to round it to 2 digits.

## • Step-by-step:

- `interest = x / 100 + 1`
- `compounded_interest = interest ** 8`
- `new_balance = 100 * compounded_interest`
- `new_balance_rounded = round(new_balance, 2)`
- `print(new_balance_rounded)`

## • Trade-off

- Less complex, but creating a lot of names

## • Inside out:

- `print(round(100*(x/100+1)**8, 2))`

## • Trade-off

- May-be more complex, better overview

# Question: What does the following program do?

- X and Y are both integers what does the following program do?

$X = X + Y$

$Y = X - Y$

$X = X - Y$

# Answer: What does the following program do?

- X and Y are both integers what does the following program do?

$X = X + Y$

$Y = X - Y$

$X = X - Y$

- This is a very abstract question
- Tip: sometimes it helps to just say what happens if  $X=1$  and  $Y=2$
- This can be a very good strategy to help you understand code